



Objetivo: desarrollar una solución que permita llevar adelante el control del estado de los vehículos.

Alumno: Agustin Feijoo

Profesoras: María Alegre y Victoria Carusso

Materia: Técnicas Avanzadas de Programación

Visión y alcance

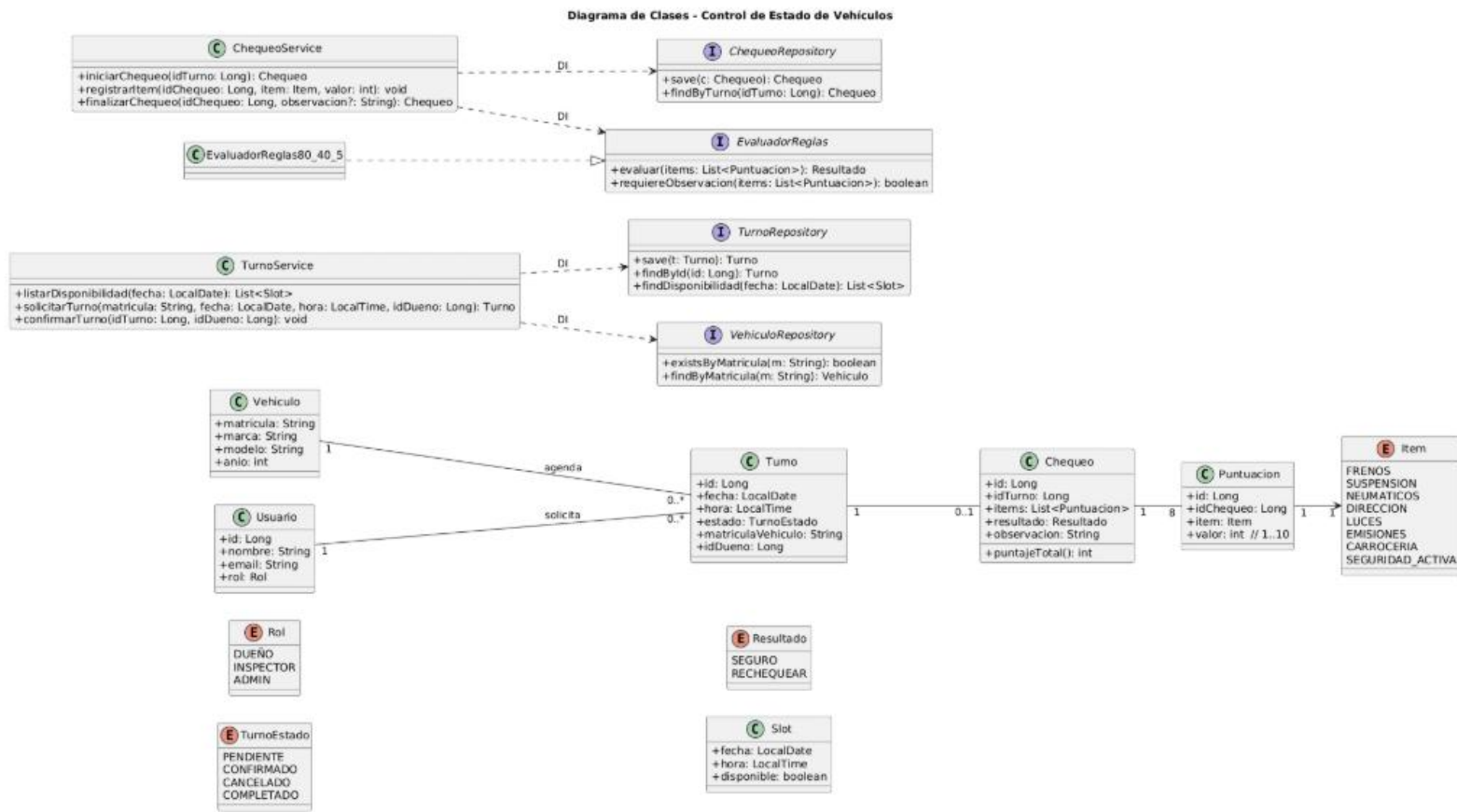
Objetivo: sistema para **solicitar turnos de revisión anual y registrar el chequeo técnico** de un vehículo, con **evaluación de 8 puntos** (1–10 cada uno) y **reglas de decisión:**

- **≥ 80 puntos totales** → “Seguro”.
- **< 40 puntos totales** → “Rechequear” + **observación obligatoria** para el dueño.
- **Cualquier punto < 5** → “Rechequear” (aunque el total sea superior).
El flujo cubre: solicitud de turno (por matrícula), confirmación, chequeo por usuario con rol “Inspector”, puntuación de 8 ítems y resultado final.

Actores:

- **Dueño** (propietario): solicita y confirma turno; consulta estado y observaciones.
 - **Inspector:** realiza el chequeo y carga las puntuaciones/observaciones.
 - **Admin** (opcional): gestiona catálogos y agendas.
-

1) Diagrama de clases (POO + DI entre objetos)



2) Modelo de datos (ER)



3) Tecnologías a utilizar

- Frontend (cliente): SPA (React/Vite) o Mobile (Flutter). Consume solo API REST (JSON).
- Backend (servidor): Java Spring Boot (Web, Validation, Security, JPA) o Node.js Express (a elección, manteniendo el desacople).
- Base de datos: PostgreSQL.
- Autenticación/Autorización: JWT con roles (DUEÑO, INSPECTOR, ADMIN).
- Inyección de dependencias: por constructor (Spring DI si Java; manual si Node).
- Especificación de API: OpenAPI (para contrato FE/BE).
-

4) Tipo de testeo y módulos a testear

- **Unitarias:**
 - EvaluadorReglas80_40_5 (casos: total ≥ 80 sin $< 5 \Rightarrow$ SEGURO; total $< 40 \Rightarrow$ RECHEQUEAR+observación; cualquier ítem $< 5 \Rightarrow$ RECHEQUEAR).
 - ChequeoService.registrarItem() y finalizarChequeo() (exactamente 8 ítems, rango 1..10, observación obligatoria cuando corresponde).
 - TurnoService.listarDisponibilidad(), solicitarTurno(), confirmarTurno().
- **Integración (API $\leftrightarrow$ Servicio $\leftrightarrow$ Repositorio $\leftrightarrow$ DB):**
 - CRUD de Turno, Chequeo y Puntuacion con PostgreSQL.
 - Autorización por rol (JWT).
- **Contrato/API:**
 - Requests/Responses válidas contra **OpenAPI/JSON Schema**.
- **E2E (mínimos):**
 - Flujo feliz: solicitar+confirmar turno $\rightarrow$ chequeo con 8 ítems $\rightarrow$ resultado SEGURO.
 - Casos límite: total 39 (requiere observación), un ítem 4 con total alto (ECHEQUEAR).

5) Justificación de tecnología y consideraciones

- API REST + cliente separado: garantiza desacople total FE/BE, escalabilidad y posibilidad de múltiples clientes (web/mobile).
- Java Spring Boot / Node Express: ecosistemas maduros para DI, validación, seguridad y pruebas; elección según expertise del equipo.
- PostgreSQL: relacional, soporta constraints y transacciones para asegurar consistencia (8 ítems, rangos 1..10, unicidad por ítem).
- DI por constructor (DIP): servicios dependen de interfaces (repositorios, evaluador de reglas); permite cambiar implementación sin tocar la lógica (ej.: memoria → DB).
- OpenAPI: evita acoplamientos implícitos y facilita pruebas de contrato y mocks de FE.
- Calidad: tests unitarios de reglas (núcleo del negocio), integración con DB, y verificación de contrato; cobertura prioritaria en ChequeoService y EvaluadorReglas.